Hypernetwork approach to Bayesian MAML

P. Borycki, P. Kubacki, M. Przewięźlikowski, T. Kuśmierczyk, J. Tabor, P. Spurek
Faculty of Mathematics and Computer Science,
Jagiellonian University, Kraków, Poland
przemyslaw.spurek@gmail.com

Abstract

The main goal of Few-Shot learning algorithms is to enable learning from small amounts of data. One of the most popular and elegant Few-Shot learning approaches is Model-Agnostic Meta-Learning (MAML). The main idea behind this method is to learn the shared universal weights of a meta-model, which are then adapted for specific tasks. However, the method suffers from over-fitting and poorly quantifies uncertainty due to limited data size. Bayesian approaches could, in principle, alleviate these shortcomings by learning weight distributions in place of point-wise weights. Unfortunately, previous modifications of MAML are limited due to the simplicity of Gaussian posteriors, MAML-like gradient-based weight updates, or by the same structure enforced for universal and adapted weights.

In this paper, we propose a novel framework for Bayesian MAML called BayesianHMAML, which employs Hypernetworks for weight updates. It learns the universal weights point-wise, but a probabilistic structure is added when adapted for specific tasks. In such a framework, we can use simple Gaussian distributions or more complicated posteriors induced by Continuous Normalizing Flows.

1 Introduction

Few-Shot learning models easily adapt to previously unseen tasks based on a few labeled samples. One of the most popular and elegant among them is Model-Agnostic Meta-Learning (MAML) [14]. The main idea behind this method is to produce universal weights which can be rapidly updated to solve new small tasks (see the first plot in Fig. 1). However, limited data sets lead to two main problems. First, the method tends to overfit to training data, preventing us from using deep architectures with large numbers of weights. Second, it lacks good quantification of uncertainty, e.g., the model does not know how reliable its predictions are. Both problems can be addressed by employing Bayesian Neural Networks (BNNs) [26], which learn distributions in place of point-wise estimates.

There exist a few Bayesian modifications of the classical MAML algorithm. Bayesian MAML [54], Amortized bayesian meta-learning [38], PACOH [41, 40], FO-MAML [30], MLAP-M [1], Meta-Mixture [22] learn distributions for the common universal weights, which are then updated to per-task local weights distributions. The above modifications of MAML, similar to the original MAML, rely on gradient-based updates. Weights specialized for small tasks are obtained by taking a fixed number of gradient steps from the standard universal weights. Such a procedure needs two levels of Bayesian regularization and the universal distribution is usually employed as a prior for the per-task specializations (see the second plot in Fig. 1). However, the hierarchical structure complicates the optimization procedure and limits updates in the MAML procedure.

The paper presents BayesianHMAML – a new framework for Bayesian Few-Shot learning. It simplifies the explained above weight-adapting procedure and thanks to the use of hypernetworks, enables learning more complicated posterior updates. Similar to the previous approaches, the final weight posteriors are obtained by updating from the universal weights. However, we avoid

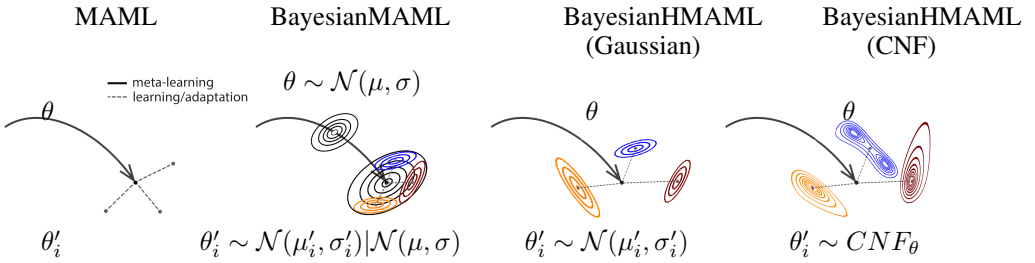

Figure 1: Comparison of four models: MAML [14], BayesianMAML [54], as well as BayesianHMAML-G, and BayesianHMAML-CNF. In the classic MAML, we have universal weights θ, which are adapted to θ'_i for individual tasks $\mathcal{T}_i$. In BayesianMAML, the posterior distributions for individual small tasks are obtained in a few gradient-based updates from the universal distribution. In BayesianHMAML-G, we learn point-wise universal weights similar to MAML, but parameters of the specialized Gaussian posteriors are produced by a hypernetwork. Unlike BayesianMAML, the per-task distributions do not share a common prior distribution. In BayesianHMAML-CNF, the hypernetwork conditions a CNF, which can model arbitrary non-Gaussian posteriors.

learning the aforementioned hierarchical structure by point-wise modeling of the universal weights. The probabilistic structure is added only later when specializing the model for a specific task. In BayesianHMAML updates from the universal weights to the per-task specialized ones are generated by hypernetworks instead of the previously used gradient-based optimization. Because hypernetworks can easily model more complex structures, they allow for better adaptations. In particular, we tested the standard Gaussian posteriors (see the third plot in Fig. 1) against more general posteriors induced by Continuous Normalizing Flows (CNF) [19] (see the right-most plot in Fig. 1).

To the best of our knowledge, BayesianHMAML is the first approach that uses hypernetworks with Bayesian learning for Few-Shot learning tasks. Our contributions can be summarized as follows:

- We introduce a novel framework for Bayesian Few-Shot learning, which simplifies updating procedure and allows using complicated posterior distributions.
- Compared to the previous Bayesian modifications of MAML, BayesianHMAML employs the hypernetworks architecture for producing significantly more flexible weight updates.
- We implement two versions of the model: BayesianHMAML-G, a classical Gaussian posterior and a generalized BayesianHMAML-CNF, relying on Conditional Normalizing Flows.

2 Background

This section introduces all the notions necessary for understanding our method. We start by presenting the background and notation for Few-Shot learning. Then, we describe how the MAML algorithm works and introduce the general idea of Hypernetworks dedicated to MAML updates. Finally, we briefly explain Conditional Normalizing Flows.

The terminology describing the Few-Shot learning setup is dispersive due to the colliding definitions used in the literature. Here, we use the nomenclature derived from the Meta-Learning literature, which is the most prevalent at the time of writing [50, 45].

Let $\mathcal{S} = \{(\mathbf{x}_l, \mathbf{y}_l)\}_{l=1}^{L}$ be a support-set containing L input-output pairs with classes distributed uniformly. In the *One-Shot* scenario, each class is represented by a single example, and $L = K$, where K is the number of the considered classes in the given task. In the *Few-Shot* scenarios, each class usually has from 2 to 5 representatives in the support set $\mathcal{S}$.

Let $\mathcal{Q} = \{(\mathbf{x}_m, \mathbf{y}_m)\}_{m=1}^{M}$ be a query set (sometimes referred to in the literature as a target set), with examples of M, where M is typically an order of magnitude greater than K. Support and query sets are grouped in task $\mathcal{T} = \{\mathcal{S}, \mathcal{Q}\}$. Few-Shot models have randomly selected examples from the training set $\mathcal{D} = \{\mathcal{T}_n\}_{n=1}^{N}$ during training. During inference, we consider task $\mathcal{T}_* = \{\mathcal{S}_*, \mathcal{X}_*\}$, where

$\mathcal{S}_*$ is a set of support with known classes and $\mathcal{X}_*$ is a set of unlabeled query inputs. The goal is to predict the class labels for the query inputs $\mathbf{x} \in \mathcal{X}_*$, assuming support set $\mathcal{S}_*$ and using the model trained on the data $\mathcal{D}$.

Model-Agnostic Meta-Learning (MAML) MAML [14] is one of the standard algorithms for Few-Shot learning, which learns the parameters of a model so that it can adapt to a new task in a few gradient steps. For the model, we use a neural network f_θ parameterized by weights θ. Its architecture consists of a feature extractor (backbone) $E(\cdot)$ and a fully connected layer. The *universal weights* $\theta = (\theta^E, \theta^H)$ include θ^E for the feature extractor and θ^H for the classification head.

When adapting for a new task $\mathcal{T}_i = \{\mathcal{S}_i, \mathcal{Q}_i\}$, the parameters θ are updated to θ'_i. Such an update is achieved in one or more gradient descent updates on $\mathcal{T}_i$. In the simplest case of one gradient update, the parameters are updated as follows:

$$\theta'_i = \theta - \alpha \nabla_\theta \mathcal{L}_{\mathcal{T}_i}(f_\theta),$$

where α is a step size hyperparameter. The loss function for a data set $\mathcal{D}$ is cross-entropy. The meta-optimization across tasks is performed via stochastic gradient descent (SGD):

$$\theta \leftarrow \theta - \beta \nabla_\theta \sum_{\mathcal{T}_i \sim p(T)} \mathcal{L}_{\mathcal{T}_i}(f_{\theta'_i})$$

where β is the meta step size (see Fig. 1).

Hypernetwork approche to MAML. HyperMAML [33] is a generalization of the MAML algorithm, which uses non-gradient-based updates generated by hypernetworks [20]. Analogically to MAML, it considers a model represented by a function f_θ with parameters θ. When adapting to a new task $\mathcal{T}_i$, the parameters of the model θ become θ'_i. Contrary to MAML, in HyperMAML the updated parameters θ'_i are computed using a hypernetwork H_ϕ as

$$\theta'_i = \theta + H_\phi(S_i, \theta).$$

The hypernetwork H_ϕ is a neural network consisting of a feature extractor $E(\cdot)$, which transforms support sets into a lower-dimensional representation, and fully connected layers aggregate the representation. To achieve permutation invariance, the embeddings are sorted according to their respective classes before aggregation.

Similarly to MAML, the universal weights θ consist of the features extractor's weights θ_E and the classification head's weights θ_H, i.e., $\theta = (\theta_E, \theta_H)$. However, HyperMAML keeps θ_E shared between tasks and updates only θ_H, e.g.,

$$\theta'_i = (\theta'^E_i, \theta'^H_i) = (\theta^E_i, \theta^H_i + H_\phi(S_i, \theta)).$$

Continuous Normalizing Flows (CNF). The idea of normalizing flows [11] relies on the transformation of a simple prior probability distribution P_Z (usually a Gaussian one) defined in the latent space Z into a complex one in the output space Y through a series of invertible mappings: $F_\eta = F_K \circ \ldots \circ F_1 : Z \to Y$. The log-probability density of the output variable is given by the change of variables formula

$$\log P_Y(y; \eta) = \log P_Z(z) - \sum_{k=1}^{K} \log \left| \det \frac{\partial F_k}{\partial z_{k-1}} \right|,$$

where $z = F_\eta^{-1}(y)$ and $P(y; \eta)$ denotes the probability density function induced by the normalizing flow with parameters η. The intermediate layers F_i must be designed so that both the inverse map and the determinant of the Jacobian are computable.

The continuous normalizing flow [8] is a modification of the above approach, where instead of a discrete sequence of iterations, we allow the transformation to be defined by a solution to a differential equation $\frac{\partial z(t)}{\partial t} = g(z(t), t)$, where g is a neural network that has an unrestricted architecture. CNF, $F_\eta : Z \to Y$, is a solution of differential equations with the initial value problem $z(t_0) = y$, $\frac{\partial z(t)}{\partial t} = g_\eta(z(t), t)$. In such a case, we have

$$F_\eta(z) = F_\eta(z(t_0)) = z(t_0) + \textstyle\int_{t_0}^{t_1} g_\eta(z(t), t) dt,$$

$$F_\eta^{-1}(y) = y + \textstyle\int_{t_1}^{t_0} g_\eta(z(t), t) dt,$$

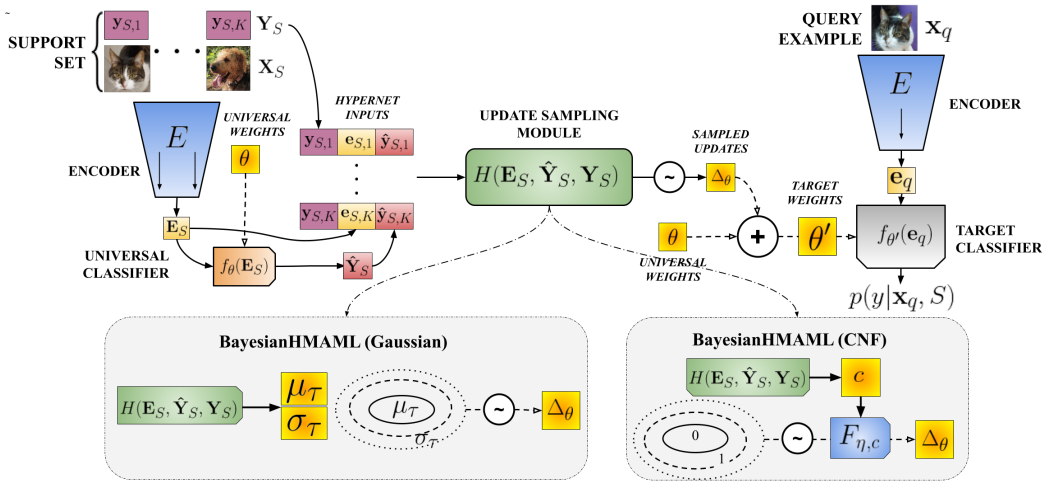

Figure 2: BayesianHMAML: Instead of updating the classifier weights with gradient descent, we use a hypernetwork H to aggregate information from the support set $\mathcal{S}$ and to produce posterior parameters. First, the support set is transformed by an encoder. The embedded symbols $\mathbf{E}_\mathcal{S}$ are then concatenated with the original labels and predictions given by the universal weights. This representation is passed to a hypernetwork $H(\mathbf{E}_\mathcal{S}, \hat{\mathbf{Y}}_\mathcal{S}, \mathbf{Y}_\mathcal{S})$ to produce posterior distributions. In our work, we consider two posterior variants: Gaussian and CNF-based. Using the hypernetwork for weight updates θ allows for larger and smarter adaptations of the posterior parameters. In the end, we sample weight updates $\Delta\theta$ from the posterior distribution to obtain weights $\theta' = \theta + \Delta\theta$ dedicated to a specific task.

where g_η defines the continuous-time dynamics of the flow F_η and $z(t_1) = y$.

The logarithmic probability of $y \in Y$ can be calculated by:

$$\log P_Y(y;\theta) = \log P_Z(F_\theta^{-1}(y)) - \int_{t_0}^{t_1} \mathrm{Tr}\left(\frac{\partial g_\theta}{\partial z(t)}\right) dt.$$

The main advantage of normalizing flows (both discrete and continuous ones) is that they are not restricted to any predefined class of distributions, e.g. Gaussian densities. For example, flow models can reliably describe densities of high-dimensional image data [23].

3 BayesianHMAML – a new framework for Bayesian learning

In this section, we present BayesianHMAML – a Bayesian extension of the classical MAML. The most straightforward Bayesian treatment for such a model is to pose priors for the model parameters and learn their posteriors. In particular, for MAML one needs to learn posterior distributions for both θ and θ'. This naturally hints towards a hierarchical Bayesian model: $\theta \to \theta'_i \to \mathcal{T}_i$, which was previously proposed in [38, 7]. Hence, variational inference along with reparametrization gradients (i.e. Bayes by backpropagation [5]) is typically used, and the following objective (evidence lower bound) is maximized with respect to variational parameters λ_i and ψ:

$$\mathcal{L}_\mathcal{D} =$$

$$\mathbb{E}_{q(\theta|\psi)}\underbrace{\left[\sum_i^N \mathbb{E}_{q(\theta'_i|\lambda_i)}\left[\log p(\mathcal{T}_i|\theta'_i) - KL\left(q(\theta'_i|\lambda_i)\|p(\theta'_i|\theta)\right)\right]\right]}_{\mathcal{L}_{\mathcal{T}_i}} - KL(q(\theta|\psi)\|p(\theta))$$

where $q(\theta'_i|\lambda_i)$ and $q(\theta|\psi)$ are respectively per-task posterior approximation and approximate posterior for the universal weights. They are tied together by the prior $p(\theta'_i|\theta)$.

The above formulation poses some challenges. First of all, updates are limited by the posterior of universal weights. Furthermore, the same distribution is used for the posterior weights for universal and updated weights. Finally, the Gaussian distribution is used for both posteriors.

Learned objective. We propose an alternative approach that alleviates the problems of previous attempts at Bayesian MAML. Contrary to them, we do not learn distributions for universal parameters θ, but instead learn them in a pointwise manner. Distributional posteriors we learn only for individual task-specialized θ'_i, where we assume their independence. Furthermore, we remove the coupling prior between θ and θ'_i, and finally, we propose a basic non-hierarchical prior $p(\theta'_i)$ instead.

BayesianHMAML's learning objective takes the following form:

$$\mathcal{L}_D^{our} = \sum_i^N \mathbb{E}_{q(\theta'_i|\lambda_i(\theta,\mathcal{S}_i))}\big[\log p(\mathcal{T}_i|\theta'_i) - \gamma \cdot KL\left(q(\theta'_i|\theta,\lambda_i(\theta,\mathcal{S}_i))\|p(\theta'_i)\right)\big],$$

where we used the standard normal priors for the weights of the neural network f, i.e., $p(\theta'_i) = \mathcal{N}(\theta'_i|0,\mathbb{I})$. The hyperparameter γ allows controlling the impact of the priors and compensating for model misspecification. Overall, the proposed modifications enable better optima for the objective and simplify the optimization landscape helping convergence.

Treatment of parameters. BayesianHMAML is a generalization of HyperMAML. Here however the weight updates result in posterior distributions instead of point-wise weights. In particular, when adapting a function f_θ with parameters θ to a task $\mathcal{T}_i$ the updated model's parameters

$$\theta'_i \sim q(\theta|\lambda_i(\theta, S_i)).$$

In BayesianHMAML the parameters λ_i are modeled by a hypernetwork as

$$\lambda_i(\theta, S_i) := H_\phi(\theta, S_i).$$

The hypernetwork H_ϕ takes support set S_i and universal weights θ and when combined with the universal weights θ produces the posterior distribution q. Thanks to the hypernetwork, we obtain unconstrained updates and can model arbitrary posterior distributions, potentially improving over all previous non-hypernetwork models. Furthermore, the hypernetwork H has a fixed number of parameters, regardless of how many tasks $\mathcal{T}_i$ are used for training. The amortized learning scheme has twofold benefits: (1) faster training; (2) regularization of learned parameters through shared architecture and common weights ϕ.

We implemented two variants of BayesianHMAML: one using the standard Gaussian posterior and a generalized one with a flow-based posterior distribution.

Gaussian version. BayesianHMAML-G is a simple realization of BayesianHMAML. In this approach, the hypernetwork H_ϕ returns the mean update and covariance matrix of a Gaussian posterior:

$$(\mu_\theta(\mathcal{S}_i), \sigma_\theta(\mathcal{S}_i)) := H_\phi(\mathcal{S}_i, \theta).$$

Weights are then sampled from the induced posterior:

$$\theta'_i \sim \mathcal{N}(\theta + \mu_\theta(\mathcal{S}_i), \sigma_\theta(\mathcal{S}_i)),$$

We apply here the mean-field assumption, but note the standard deviations σ are not entirely independent. Due to the used amortization scheme, they are tied together and to the means μ by shared weights ϕ of the hypernetwork. Posterior means however are additionally explicitly dependent by the universal weights θ. Like the classic MAML, any change of θ affects all the values of θ'_i.

CNF version. BayesianHMAML-CNF is a generalization of BayesianHMAML-G, where we use conditional flows to produce weight posteriors for the specialized tasks. Similar to BayesianHMAML-G, we employ a hypernetwork to amortize updates of the target model parameters. However, in the above model the hypernetwork outputs parameters of a Gaussian distribution, whereas in BayesianHMAML-CNF it is responsible for conditioning a flow $F_{\eta,C(\theta,\mathcal{S}_i)}(\cdot)$(see Fig. 2 for comparison):

$$C(\theta, \mathcal{S}_i) := H_\phi(\mathcal{S}_i, \theta),$$

The conditioning vector C is added to each layer of the flow to parameterize function $g_{\theta,C}$, so in the end, the flow $F_{\eta,C}$ depends on trainable parameters η and conditioning parameters C. Then, the posterior for a task $\mathcal{T}_i$ is obtained by a two-stage process:

$$\Delta\theta_i' \sim F_{\eta,C(\theta,\mathcal{S}_i)}$$
$$\theta_i' = \theta + \Delta\theta_i',$$

where the shape of the posterior distribution is determined by the flow F, but its position, similarly to BayesianHMAML-G, mainly by the universal weights. From the implementation point of view, sampling from the conditioned flow also happens in two stages. First, we sample some z from a flow prior and then, push this z through a chain of deterministic transformations to obtain the final sample. Formally,

$$\Delta\theta_i' := F_{\eta,C(\theta,\mathcal{S}_i)}(z), \text{ where } z \sim \mathrm{N}(0, t \cdot \mathbb{I}),$$

where t is a hyperparameter, we used $t = 0.1$.

Architecture of BayesianHMAML. The goal here is to predict the class distribution $p(\mathbf{y}|x_q, \mathcal{S})$, given a single query example $\mathbf{x}_q$, and a set of support examples $\mathcal{S}$. The architecture of BayesianHMAML is illustrated in Fig. 2. Following MAML, we consider a parametric function f_θ, which models the discriminative distribution for the classes. In addition, our architecture consists of a trainable encoding network $E(\cdot)$, which transforms data into a low-dimensional representation. The predictions are then calculated following

$$p(\mathbf{y}|x_q, \theta') = f_{\theta'}(\mathbf{e}_q),$$

where $\mathbf{e}_q$ is the query example $\mathbf{x}_q$ transformed using encoder $E(\cdot)$, and θ' come from the posterior distribution for a considered task (either in BayesianHMAML-G or BayesianHMAML-CNF variant). In contrast to the MAML gradient-based adaptations, we predict weights directly from the support set using a hypernetwork. The hypernetwork observes support examples with the corresponding true labels and decides how the global parameters θ should be adjusted for a considered task. The two possible variants: BayesianHMAML-G and BayesianHMAML-CNF are illustrated (denoted by gray squares) in Fig. 2.

In BayesianHMAML, parameters of the learned posterior distribution are obtained by the hypernetwork $H_\phi(\theta, \mathcal{S})$. First, each of the inputs from support set $\mathcal{S}$ is transformed by Encoder $E(\cdot)$ to obtain low-dimensional matrix of embeddings $\mathbf{E}_\mathcal{S} = [\mathbf{e}_{\mathcal{S},1}, \ldots, \mathbf{e}_{\mathcal{S},K}]^\mathrm{T}$. Next, the corresponding class labels for support examples, $\mathbf{Y}_\mathcal{S} = [\mathbf{y}_{\mathcal{S},1}, \ldots, \mathbf{y}_{\mathcal{S},K}]^\mathrm{T}$ are concatenated to the corresponding embeddings stored in the matrix $\mathbf{E}_\mathcal{S}$. Furthermore, we calculate the predicted values for the examples in the support set using the general model as $f_\theta(\mathbf{E}_\mathcal{S}) = \hat{\mathbf{Y}}_\mathcal{S}$, and also concatenate them into $\mathbf{E}_\mathcal{S}$. The predictions of the global model f_θ help identify classification errors and correct them by weight adaptation.

Finally, the transformed support $\mathbf{E}_\mathcal{S}$, together with true labels $\mathbf{Y}_\mathcal{S}$, and the corresponding predictions $\hat{\mathbf{Y}}_\mathcal{S}$ from the model with universal weights are passed as input to the hypernetwork as $H(\mathbf{E}_\mathcal{S}, \hat{\mathbf{Y}}_\mathcal{S}, \mathbf{Y}_\mathcal{S})$, which then predicts the posterior controlling parameters. In our case, the hypernetwork consists of fully-connected layers with ReLU activations.

Implementation details. Practical learning of BayesianHMAML is performed with stochastic gradients calculated w.r.t to the universal weights θ (i.e., $\nabla_\theta \mathcal{L}_\mathcal{T}^{our}$), the hypernetwork weights ϕ (i.e., $\nabla_\phi \mathcal{L}_\mathcal{T}^{our}$), and in case of BayesianHMAML-CNF also w.r.t η (i.e., $\nabla_\eta \mathcal{L}_\mathcal{T}^{our}$), all of which have fixed sizes, which do not depend on the number of tasks N. We approximate the learning objective using mini-batches as

$$\mathcal{L}_\mathcal{T}^{our} = \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \left[\frac{1}{P} \sum_{\theta_i' \sim q(\theta, \lambda_i(\theta, \mathcal{S}_i))} \left[\mathcal{L}_{\mathcal{T}_i}(f_{\theta_i'}) - \gamma KL(q(\theta_i'|\theta, \lambda_i(\theta, \mathcal{S}_i)) \| \mathcal{N}(\theta_i'|0, \mathbb{I})) \right] \right],$$

where in each iteration we sample some number of tasks from $p(\mathcal{T})$ and then, for each task, we sample $P = 5$ samples θ_i' from the posterior q. How exactly is the sampling (and reparametrization) performed, depends on whether we use BayesianHMAML-G or BayesianHMAML-CNF.

For BayesianHMAML-G the KL-divergence can be calculated in a closed form. For BayesianHMAML-CNF we use a Monte-Carlo estimate of P samples:

$$KL(\cdot) = \frac{1}{P} \sum_{z \sim \mathcal{N}(0, t \cdot \mathbb{I})} \left(\log \mathcal{N} \left(F_{\eta,C}^{-1}(\Delta\theta_i'|0, t \cdot \mathbb{I} \right) + \log \det |J| - \log N(\theta_i'|0, \mathbb{I}) \right),$$

where $\Delta\theta_i' \equiv F_{\eta,C}(z)$ and J is the flow transition Jacobian. Contrary to non-amortized methods, BayesianHMAML is easy to maintain (and scales well) since we need to store only a fixed number of parameters $\{\theta, \psi\}$ (or in case of BayesianHMAML-CNF: $\{\theta, \psi, \eta\}$). Finally, for the hyperparameter γ, we apply an annealing scheme [6]: the parameter γ grows from zero to a fixed constant during training. The final value γ_{max} is a hyperparameter of the model.

4 Related Work

The problem of Meta-Learning and Few-Shot learning [21, 43, 4] is currently one of the most important topics in deep learning, with the abundance of methods emerging as a result. They can be roughly categorized into three groups: Model-based methods, Metric-based methods, Optimization-based methods. In all these groups, we can find methods that use Hypernetworks and Bayesian learning (but not both at the same time). We briefly review the approaches below.

Model-based methods aim to adapt to novel tasks quickly by utilizing mechanisms such as memory [39, 27, 56], Gaussian Processes [37, 32, 51, 44], or generating fast weights based on the support set with set-to-set architectures [34, 3, 52, 57]. Other approaches maintain a set of weight templates and, based on those, generate target weights quickly through gradient-based optimization such as [55]. The fast weights approaches can be interpreted as using Hypernetworks [20] – models which learn to generate the parameters of neural networks performing the designated tasks.

Metric-based methods learn a transformation to a feature space where the distance between examples from the same class is small. The earliest examples of such methods are Matching Networks [49] and Prototypical Networks [47]. Subsequent works show that metric-based approaches can be improved by techniques such as learnable metric functions [48], conditioning the model on tasks [31] or predicting the parameters of the kernel function to be calculated between support and query data with Hypernetworks [45]. In [42], authors introduce a meta-learning technique that uses a generative parameter model to capture the diverse range of parameters useful for distribution over tasks.

Table 1: Classification accuracy for inference on CUB and $mini-ImageNet$ data sets in the 1-shot and 5-shot settings. The highest results are in bold and the second-highest in italic.

	CUB		mini-ImageNet	
Method	1-shot	5-shot	1-shot	5-shot
Feature Transfer [58]	46.19 ± 0.64	68.40 ± 0.79	39.51 ± 0.23	60.51 ± 0.55
ProtoNet [47]	52.52 ± 1.90	75.93 ± 0.46	44.19 ± 1.30	64.07 ± 0.65
MAML [14]	56.11 ± 0.69	74.84 ± 0.62	45.39 ± 0.49	61.58 ± 0.53
MAML++ [2]	–	–	52.15 ± 0.26	$\mathit{68.32 \pm 0.44}$
FEAT [52]	$\mathbf{68.87 \pm 0.22}$	$\mathbf{82.90 \pm 0.15}$	$\mathbf{55.15 \pm 0.20}$	$\mathbf{71.61 \pm 0.16}$
LLAMA [18]	–	–	49.40 ± 1.83	–
VERSA [16]	–	–	48.53 ± 1.84	67.37 ± 0.86
Amortized VI [16]	–	–	44.13 ± 1.78	55.68 ± 0.91
DKT + BNCosSim [32]	62.96 ± 0.62	77.76 ± 0.62	49.73 ± 0.07	64.00 ± 0.09
VAMPIRE [29]	–	–	51.54 ± 0.74	64.31 ± 0.74
ABML [38]	49.57 ± 0.42	68.94 ± 0.16	45.00 ± 0.60	–
OVE PG GP + Cosine (ML) [46]	63.98 ± 0.43	77.44 ± 0.18	50.02 ± 0.35	64.58 ± 0.31
OVE PG GP + Cosine (PL) [46]	60.11 ± 0.26	79.07 ± 0.05	48.00 ± 0.24	67.14 ± 0.23
Bayesian MAML [54]	55.93 ± 0.71	–	$\mathit{53.80 \pm 1.46}$	64.23 ± 0.69
HyperMAML [33]	66.11 ± 0.28	78.89 ± 0.19	51.84 ± 0.57	66.29 ± 0.43
BayesianHMAML (G)	66.57 ± 0.47	79.86 ± 0.31	52.54 ± 0.46	67.39 ± 0.35
BayesianHMAML (G)+adapt.	$\mathit{66.92 \pm 0.38}$	$\mathit{80.47 \pm 0.38}$	52.69 ± 0.38	68.24 ± 0.47
BayesianHMAML (CNF)	61.55 ± 0.69	75.41 ± 0.21	49.39 ± 0.33	64.77 ± 0.21
BayesianHMAML (CNF)+adapt.	62.15 ± 0.51	75.69 ± 0.32	49.61 ± 0.24	65.48 ± 0.43

Optimization-based methods such as MetaOptNet [24] is based on the idea of an optimization process over the support set within the Meta-Learning framework. Arguably, the most popular of this family of methods is Model-Agnostic Meta-Learning (MAML) [14]. In literature, we have various techniques for stabilizing its training and improving performance, such as Multi-Step Loss Optimization [2], or using the Bayesian variant of MAML [54].

Due to a need for calculating second-order derivatives when computing the gradient of the meta-training loss, training the classical MAML introduces a significant computational overhead. The authors show that in practice, the second-order derivatives can be omitted at the cost of small gradient estimation error and minimally reduced accuracy of the model [14, 30]. Methods such as iMAML

Table 2: Classification accuracy for inference on cross-domain data sets (Omniglot→EMNIST and mini-ImageNet→CUB), in the 1-shot and 5-shot settings. The highest results are marked in **bold** and the second-highest in *italics*.

Method	Omniglot→EMNIST		mini-ImageNet→CUB	
	1-shot	5-shot	1-shot	5-shot
Feature Transfer [58]	64.22 ± 1.24	86.10 ± 0.84	32.77 ± 0.35	50.34 ± 0.27
ProtoNet [47]	72.04 ± 0.82	87.22 ± 1.01	33.27 ± 1.09	52.16 ± 0.17
MAML [14]	74.81 ± 0.25	83.54 ± 1.79	34.01 ± 1.25	48.83 ± 0.62
DKT [32]	75.40 ± 1.10	$\mathbf{90.30 \pm 0.49}$	$\mathbf{40.14 \pm 0.18}$	$\mathit{56.40 \pm 1.34}$
OVE PG GP + Cosine (ML) [46]	68.43 ± 0.67	86.22 ± 0.20	39.66 ± 0.18	55.71 ± 0.31
OVE PG GP + Cosine (PL) [46]	77.00 ± 0.50	87.52 ± 0.19	$\mathit{37.49 \pm 0.11}$	$\mathbf{57.23 \pm 0.31}$
Bayesian MAML [54]	63.94 ± 0.47	65.26 ± 0.30	33.52 ± 0.36	51.35 ± 0.16
HyperMAML [33]	79.07 ± 1.09	89.22 ± 0.78	36.32 ± 0.61	49.43 ± 0.14
BayesianHMAML (G)	$\mathit{80.95 \pm 0.46}$	89.21 ± 0.27	36.90 ± 0.34	49.24 ± 0.38
BayesianHMAML (G)+adapt.	$\mathbf{81.05 \pm 0.47}$	$\mathit{89.76 \pm 0.26}$	37.23 ± 0.44	50.79 ± 0.59
BayesianHMAML (CNF)	72.02 ± 0.56	82.36 ± 0.12	33.77 ± 0.30	44.09 ± 0.32
BayesianHMAML (CNF)+adapt.	72.54 ± 0.36	82.63 ± 0.37	34.67 ± 0.35	45.14 ± 0.27

and Sign-MAML propose to solve this issue with implicit gradients or Sign-SGD optimization [36, 12]. The optimization process can also be improved by training the base initialization [28, 35]. Furthermore, gradient-based optimization for few-shot tasks can be discarded altogether in favor of updates generated by hypernetworks [33].

Classical MAML-based algorithms have problems with over-fitting. To address this problem, we can use the Bayesian models [38, 54, 18, 22, 29]. In practice, the Bayesian model contains two levels of probability distribution on weights. We have Bayesian universal weights, which are updated for different tasks [18]. Its leads to a hierarchical Bayes formulation. Bayesian networks perform better in few-shot settings and reduce over-fitting. Several variants of the hierarchical Bayes model have been proposed based on different Bayesian inference methods [15, 54, 16, 29]. Another branch of probabilistic methods is represented by PAC-Bayes based method [7, 1, 41, 40, 10, 13]. In the PAC-Bayes framework, we use the Gibbs error when sampling priors. But still, we have a double level of Bayesian networks.

In [42], authors introduce a meta-learning technique using a generative parameter model to capture the diverse range of parameters useful for task distribution. In VERSA [17], authors use amortization networks to produce distribution over weights directly.

In the paper, we propose BayesianHMAML-G, which uses probability distribution update only for weight dedicated to small tasks. Thanks to such a solution, we produce significantly larger updates.

5 Experiments

In our experiments, we follow the unified procedure proposed by [9]. We split the data sets into the standard train, validation, and test class subsets, used commonly in the literature [39, 9, 32]. We report the performance of both variants of BayesianHMAML averaged over three training runs for each setting.

We report results for BayesianHMAML and for the model with adaptation. In the case of BayesianHMAML + adaptation, we tune a copy of the hypernetwork on the support set separately for each validation task. This way, we ensure that our model does not take unfair advantage of the validation tasks. In the case of hypernetwork-based approaches adaptation is a common strategy introduced by [45].

First, we consider a classical Few-Shot learning scenario on two data sets: Caltech-USCD Birds (CUB) and mini-ImageNet.

In the case of CUB, BayesianHMAML-G obtains the second-best score in the 1-shot and 5-shot settings. In the case of $mini - ImageNet$, we report comparable results comparable to other methods. We emphasize that BayesianHMAML obtains the best score in the area of Bayesian [18, 16, 32, 38, 46, 54] and MAML-based methods [14, 33], save for MAML++ [2].

In the cross-domain adaptation setting, the model is evaluated on tasks from a different distribution than the one on which it had been trained. We report the results in Table 2. In the task of 1-shot Omniglot→EMNIST classification, BayesianHMAML-G achieves the best result. The 5-shot Omniglot→EMNIST classification task BayesianHMAML-G yields comparable results to baseline methods. In the mini-ImageNet→CUB classification, our method performs comparably to baseline methods such as MAML and ProtoNet.

It needs to be highlighted that in all experiments BayesianHMAML-G achieves better performance than BayesianHMAML-CNF. It is caused mainly by the fact that the Gaussian posterior, whenever needed, can easily degenerate to a near-point distribution. In the case of the Flow-based model, the weight distributions are more complex and learning is significantly harder.

The primary reason for using Bayesian approaches is better uncertainty quantification. Our models always give predictions for elements from support and query sets similar to the ones by HyperMAML and MAML. What is however crucial, we observe higher uncertainty in the case of elements from out of distribution. To illustrate that we trained BayesianHMAML on cross-domain adaptation setting Omniglot→EMNIST. Then, we sampled testing tasks from EMNIST during the evaluation and we sampled one thousand different weights from the distribution for our support set. Results are shown in Fig. 3.

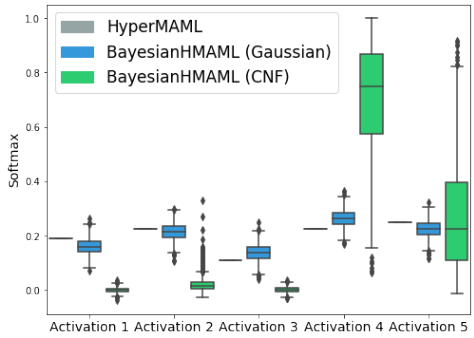

Figure 3: Predictions of HyperMAML, BayesianHMAML-G, and BayesianHMAML-CNF on out-of-distribution data. We trained the models on cross-domain data Omniglot→EMNIST in a 5-shot setting, and then, for each of them, we sampled one thousand predictions. We show results on activations of 5 classes. Note, Bayesian models exhibit high uncertainty on out-of-distribution data. Furthermore, BayesianHMAML-CNF produces the most diverse predictions.

6 Conclusions

In this work, we introduced BayesianHMAML – a novel Bayesian Meta-Learning algorithm strongly motivated by MAML. In BayesianHMAML, we have universal weights trained in a point-wise manner, similar to MAML, and Bayesian updates modeled with hypernetworks. Such an approach allows for significantly larger updates in the adaptation phase and better uncertainty quantification. Our experiments show that BayesianHMAML outperforms all Bayesian and MAML-based methods in several standard Few-Shot learning benchmarks and in most cases, achieves results better or comparable to other state-of-the-art methods. Crucially, BayesianHMAML can be used to estimate the uncertainty of the predictions, enabling possible applications in critical areas of deep learning, such as medical diagnosis or autonomous driving.

7 Appendix: Training details

In this section, we present details of the training and architecture overview.

7.1 Architecture details

Encoder For each experiment described in the main body of this work, we utilize a shallow convolutional encoder (feature extractor), commonly used in the literature [14, 9, 32]. This encoder consists of four convolutional layers, each consisting of a convolution, batch normalization, and ReLU nonlinearity. Each convolutional layer has an input and output size of 64, except for the first layer, where the input size equals the number of image channels. We also apply max-pooling between each convolution, decreasing the resolution of the processed feature maps by half. The output of the encoder is flattened to process it in the next layers.

In the case of the Omniglot and EMNIST images, such encoder compresses the images into 64-element embedding vectors, which serve as input to the Hypernetwork (with the above-described enhancements) and the classifier. However, in the case of substantially larger mini-ImageNet and CUB images, the backbone outputs a feature map of shape $[64 \times 5 \times 5]$, which would translate to 1600-element embeddings and lead to an over parametrization of the Hypernetwork and classifier which processes them and increase the computational load. Therefore, we apply an average pooling operation to the obtained feature maps and ultimately also obtain embeddings of shape 64. Thus, we can use significantly smaller Hypernetworks.

Hypernetwork The Hypernetwork transforms the enhanced embeddings of the support examples of each class in a task into the updates for the portion of classifier weights predicting that class. It consists of three fully-connected layers with ReLU activation function between each consecutive pair of layers. In the hypernetwork, we use a hidden size of 256 or 512.

Classifier The universal classifier is a single fully-connected layer with the input size equal to the encoder embedding size (in our case 64) and the output size equal to the number of classes. When using the strategy with embeddings enhancement, we freeze the classifier to get only the information about the behavior of the classifier. This means we do not calculate the gradient for the classifier in this step of the forward pass. Instead, gradient calculation for the classifier takes place during the classification of the query data.

7.2 Training details

In all of the experiments described in the main body of this work, we utilize the switch and the embedding enhancement mechanisms. We use the Adam optimizer and a multi-step learning rate schedule with the decay of 0.3 and learning rate starting from 0.01 or 0.001. We train BayesianHMAML for 4000 epochs on all the data sets, save for the simpler Omniglot $\rightarrow$ EMNIST classification task, where we train for 2048 epochs instead.

7.3 Hyperparameters

Below, we outline the hyperparameters of architecture and training procedures used in each experiment.

hyperparameter	CUB	mini-ImageNet	mini-ImageNet $\rightarrow$ CUB	Omniglot $\rightarrow$ EMNIST
learning rate	0.01	0.001	0.001	0.01
Hyper Network depth	3	3	3	3
Hyper Network width	512	256	256	512
epochs no.	4000	4000	4000	2048
milestones	$51, 550$	$101, 1100$	$101, 1100$	$51, 550$
γ	$1e-4$	$1e-4$	$1e-5$	0.001
num. of samples (train)	5	7	5	5

Table 3: Hyperparameters for each of conducted 1-shot experiments. (G)

8 Appendix: Extended results

We include an expanded version of Table 1 from the main manuscript in Table 7, comparing our approach to a larger number of meta-learning methods.

hyperparameter	CUB	mini-ImageNet	mini-ImageNet → CUB	Omniglot → EMNIST
learning rate	0.001	0.001	0.001	0.01
Hyper Network depth	3	3	3	3
Hyper Network width	256	256	256	512
epochs no.	4000	4000	4000	2048
milestones	$101, 1100$	$101, 1100$	$101, 1100$	$51, 550$
γ	$1e-5$	$1e-5$	$1e-4$	0.001
num. of samples (train)	5	5	5	5

Table 4: Hyperparameters for each of the conducted 5-shot experiments. (G)

hyperparameter	CUB	mini-ImageNet	mini-ImageNet → CUB	Omniglot → EMNIST
learning rate	0.001	0.001	0.001	0.001
Hyper Network depth	3	3	3	3
Hyper Network width	512	256	256	512
epochs no.	4000	4000	4000	2048
milestones	$51, 550$	$101, 1100$	$101, 1100$	$51, 550$
γ	$1e-6$	$1e-6$	$1e-6$	$1e-4$
num. of samples (train)	5	5	5	5

Table 5: Hyperparameters for each of conducted 1-shot experiments. (CNF)

hyperparameter	CUB	mini-ImageNet	mini-ImageNet → CUB	Omniglot → EMNIST
learning rate	0.001	0.001	0.001	0.001
Hyper Network depth	3	3	3	3
Hyper Network width	256	256	256	512
epochs no.	4000	4000	4000	2048
milestones	$101, 1100$	$101, 1100$	$101, 1100$	$51, 550$
γ	$1e-6$	$1e-6$	$1e-6$	$1e-6$
num. of samples (train)	5	5	5	5

Table 6: Hyperparameters for each of the conducted 5-shot experiments. (CNF)

References

[1] R. Amit and R. Meir. Meta-learning by adjusting priors based on extended pac-bayes theory. In *International Conference on Machine Learning*, pages 205–214. PMLR, 2018.

[2] A. Antoniou, H. Edwards, and A. Storkey. How to train your maml, 2018. URL `https://arxiv.org/abs/1810.09502`.

[3] M. Bauer, M. Rojas-Carulla, J. B. Świątkowski, B. Schölkopf, and R. E. Turner. Discriminative k-shot learning using probabilistic models, 2017.

[4] S. Bengio, Y. Bengio, J. Cloutier, and J. Gecsei. On the optimization of a synaptic learning rule. 1992.

[5] C. Blundell, J. Cornebise, K. Kavukcuoglu, and D. Wierstra. Weight uncertainty in neural network. In *International conference on machine learning*, pages 1613–1622. PMLR, 2015.

[6] S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Jozefowicz, and S. Bengio. Generating sentences from a continuous space. In *20th SIGNLL Conference on Computational Natural Language Learning, CoNLL 2016*, pages 10–21. Association for Computational Linguistics (ACL), 2016.

[7] L. Chen and T. Chen. Is bayesian model-agnostic meta learning better than model-agnostic meta learning, provably? In *International Conference on Artificial Intelligence and Statistics*, pages 1733–1774. PMLR, 2022.

[8] T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural ordinary differential equations. In *NeurIPS*, pages 6571–6583, 2018.

[9] W.-Y. Chen, Y.-C. Liu, Z. Kira, Y.-C. F. Wang, and J.-B. Huang. A closer look at few-shot classification. *arXiv preprint arXiv:1904.04232*, 2019.

Table 7: Classification accuracy for inference on CUB and $mini-ImageNet$ data sets in the 1-shot and 5-shot settings. The highest results are in bold and the second-highest in italic.

	CUB		mini-ImageNet	
Method	1-shot	5-shot	1-shot	5-shot
ML-LSTM [39]	–	–	43.44 ± 0.77	60.60 ± 0.71
SNAIL [27]	–	–	45.10	55.20
Feature Transfer [58]	46.19 ± 0.64	68.40 ± 0.79	39.51 ± 0.23	60.51 ± 0.55
ProtoNet [47]	52.52 ± 1.90	75.93 ± 0.46	44.19 ± 1.30	64.07 ± 0.65
MAML [14]	56.11 ± 0.69	74.84 ± 0.62	45.39 ± 0.49	61.58 ± 0.53
MAML++ [2]	–	–	52.15 ± 0.26	68.32 ± 0.44
FEAT [52]	$\mathbf{68.87 \pm 0.22}$	$\mathbf{82.90 \pm 0.15}$	$\mathbf{55.15 \pm 0.20}$	$\mathbf{71.61 \pm 0.16}$
LLAMA [18]	–	–	49.40 ± 1.83	–
VERSA [16]	–	–	48.53 ± 1.84	67.37 ± 0.86
Amortized VI [16]	–	–	44.13 ± 1.78	55.68 ± 0.91
Meta-Mixture [22]	–	–	49.60 ± 1.50	64.60 ± 0.92
Baseline++ [9]	61.75 ± 0.95	78.51 ± 0.59	47.15 ± 0.49	66.18 ± 0.18
MatchingNet [49]	60.19 ± 1.02	75.11 ± 0.35	48.25 ± 0.65	62.71 ± 0.44
RelationNet [48]	62.52 ± 0.34	78.22 ± 0.07	48.76 ± 0.17	64.20 ± 0.28
DKT + CosSim [32]	63.37 ± 0.19	77.73 ± 0.26	48.64 ± 0.45	62.85 ± 0.37
DKT + BNCosSim [32]	62.96 ± 0.62	77.76 ± 0.62	49.73 ± 0.07	64.00 ± 0.09
VAMPIRE [29]	–	–	51.54 ± 0.74	64.31 ± 0.74
PLATIPUS [15]	–	–	50.13 ± 1.86	–
ABML [38]	49.57 ± 0.42	68.94 ± 0.16	45.00 ± 0.60	–
OVE PG GP + Cosine (ML) [46]	63.98 ± 0.43	77.44 ± 0.18	50.02 ± 0.35	64.58 ± 0.31
OVE PG GP + Cosine (PL) [46]	60.11 ± 0.26	79.07 ± 0.05	48.00 ± 0.24	67.14 ± 0.23
FO-MAML [30]	–	–	48.70 ± 1.84	63.11 ± 0.92
Reptile [30]	–	–	49.97 ± 0.32	65.99 ± 0.58
VSM [56]	–	–	$\mathit{54.73 \pm 1.60}$	68.01 ± 0.90
PPA [34]	–	–	$\mathit{54.53 \pm 0.40}$	–
HyperShot [45]	65.27 ± 0.24	79.80 ± 0.16	52.42 ± 0.46	68.78 ± 0.29
HyperShot+ adaptation [45]	66.13 ± 0.26	80.07 ± 0.22	53.18 ± 0.45	69.62 ± 0.2
iMAML-HF [36]	–	–	49.30 ± 1.88	–
SignMAML [12]	–	–	42.90 ± 1.50	60.70 ± 0.70
Bayesian MAML [54]	55.93 ± 0.71	–	53.80 ± 1.46	64.23 ± 0.69
Unicorn-MAML [53]	–	–	54.89	–
Meta-SGD [25]	–	–	50.47 ± 1.87	64.03 ± 0.94
MetaNet [28]	–	–	49.21 ± 0.96	–
PAMELA [35]	–	–	53.50 ± 0.89	$\mathit{70.51 \pm 0.67}$
HyperMAML [33]	66.11 ± 0.28	78.89 ± 0.19	51.84 ± 0.57	66.29 ± 0.43
BayesianHMAML (G)	66.57 ± 0.47	79.86 ± 0.31	52.54 ± 0.46	67.39 ± 0.35
BayesianHMAML (G) + adaptation	$\mathit{66.92 \pm 0.38}$	$\mathit{80.47 \pm 0.38}$	52.69 ± 0.38	68.24 ± 0.47
BayesianHMAML (CNF)	61.55 ± 0.69	75.41 ± 0.21	49.39 ± 0.33	64.77 ± 0.21
BayesianHMAML (CNF) + adaptation	62.15 ± 0.51	75.69 ± 0.32	49.61 ± 0.24	65.48 ± 0.43

[10] N. Ding, X. Chen, T. Levinboim, S. Goodman, and R. Soricut. Bridging the gap between practice and pac-bayes theory in few-shot meta-learning. *Advances in Neural Information Processing Systems*, 34:29506–29516, 2021.

[11] L. Dinh, D. Krueger, and Y. Bengio. Nice: Non-linear independent components estimation. *arXiv preprint arXiv:1410.8516*, 2014.

[12] C. Fan, P. Ram, and S. Liu. Sign-maml: Efficient model-agnostic meta-learning by signsgd. *CoRR*, abs/2109.07497, 2021.

[13] A. Farid and A. Majumdar. Generalization bounds for meta-learning via pac-bayes and uniform stability. *Advances in Neural Information Processing Systems*, 34:2173–2186, 2021.

[14] C. Finn, P. Abbeel, and S. Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International Conference on Machine Learning*, pages 1126–1135. PMLR, 2017.

[15] C. Finn, K. Xu, and S. Levine. Probabilistic model-agnostic meta-learning. *Advances in neural information processing systems*, 31, 2018.

[16] J. Gordon, J. Bronskill, M. Bauer, S. Nowozin, and R. Turner. Meta-learning probabilistic inference for prediction. In *International Conference on Learning Representations*, 2018.

[17] J. Gordon, J. Bronskill, M. Bauer, S. Nowozin, and R. Turner. Meta-learning probabilistic inference for prediction. In *International Conference on Learning Representations (ICLR 2019)*. OpenReview. net, 2019.

[18] E. Grant, C. Finn, S. Levine, T. Darrell, and T. Griffiths. Recasting gradient-based meta-learning as hierarchical bayes. In *International Conference on Learning Representations*, 2018.

[19] W. Grathwohl, R. T. Chen, J. Bettencourt, I. Sutskever, and D. Duvenaud. Ffjord: Free-form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2018.

[20] D. Ha, A. Dai, and Q. V. Le. Hypernetworks. *arXiv preprint arXiv:1609.09106*, 2016.

[21] T. Hospedales, A. Antoniou, P. Micaelli, and A. Storkey. Meta-learning in neural networks: A survey. 2020.

[22] G. Jerfel, E. Grant, T. L. Griffiths, and K. Heller. Reconciling meta-learning and continual learning with online mixtures of tasks. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, pages 9122–9133, 2019.

[23] D. P. Kingma and P. Dhariwal. Glow: Generative flow with invertible 1x1 convolutions. In *NeurIPS*, pages 10215–10224, 2018.

[24] K. Lee, S. Maji, A. Ravichandran, and S. Soatto. Meta-learning with differentiable convex optimization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10657–10665, 2019.

[25] Z. Li, F. Zhou, F. Chen, and H. Li. Meta-sgd: Learning to learn quickly for few-shot learning, 2017.

[26] D. J. MacKay. A practical bayesian framework for backpropagation networks. *Neural computation*, 4(3):448–472, 1992.

[27] N. Mishra, M. Rohaninejad, X. Chen, and P. Abbeel. A simple neural attentive meta-learner. In *International Conference on Learning Representations*, 2018.

[28] T. Munkhdalai and H. Yu. Meta networks. In *International Conference on Machine Learning*, pages 2554–2563. PMLR, 2017.

[29] C. Nguyen, T.-T. Do, and G. Carneiro. Uncertainty in model-agnostic meta-learning using variational inference. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 3090–3100, 2020.

[30] A. Nichol, J. Achiam, and J. Schulman. On first-order meta-learning algorithms. *arXiv preprint arXiv:1803.02999*, 2018.

[31] B. N. Oreshkin, P. Rodriguez, and A. Lacoste. Tadam: Task dependent adaptive metric for improved few-shot learning. *arXiv preprint arXiv:1805.10123*, 2018.

[32] M. Patacchiola, J. Turner, E. J. Crowley, M. O'Boyle, and A. J. Storkey. Bayesian meta-learning for the few-shot setting via deep kernels. *Advances in Neural Information Processing Systems*, 33, 2020.

[33] M. Przewięźlikowski, P. Przybysz, J. Tabor, M. Zięba, and P. Spurek. Hypermaml: Few-shot adaptation of deep models with hypernetworks. *arXiv preprint arXiv:2205.15745*, 2022.

[34] S. Qiao, C. Liu, W. Shen, and A. Yuille. Few-shot image recognition by predicting parameters from activations, 2017.

[35] J. Rajasegaran, S. H. Khan, M. Hayat, F. S. Khan, and M. Shah. Meta-learning the learning trends shared across tasks. *CoRR*, abs/2010.09291, 2020. URL `https://arxiv.org/abs/2010.09291`.

[36] A. Rajeswaran, C. Finn, S. M. Kakade, and S. Levine. Meta-learning with implicit gradients. *Advances in Neural Information Processing Systems*, 32:113–124, 2019.

[37] C. E. Rasmussen. Gaussian processes in machine learning. In *Summer school on machine learning*, pages 63–71. Springer, 2003.

[38] S. Ravi and A. Beatson. Amortized bayesian meta-learning. In *International Conference on Learning Representations*, 2018.

[39] S. Ravi and H. Larochelle. Optimization as a model for few-shot learning. In *ICLR*, 2017.

[40] J. Rothfuss, M. Josifoski, and A. Krause. Meta-learning bayesian neural network priors based on pac-bayesian theory. 2020.

[41] J. Rothfuss, V. Fortuin, M. Josifoski, and A. Krause. Pacoh: Bayes-optimal meta-learning with pac-guarantees. In *International Conference on Machine Learning*, pages 9116–9126, 2021.

[42] A. A. Rusu, D. Rao, J. Sygnowski, O. Vinyals, R. Pascanu, S. Osindero, and R. Hadsell. Meta-learning with latent embedding optimization. In *International Conference on Learning Representations*, 2018.

[43] J. Schmidhuber. Learning to Control Fast-Weight Memories: An Alternative to Dynamic Recurrent Networks. *Neural Computation*, 4(1):131–139, 01 1992. ISSN 0899-7667.

[44] M. Sendera, J. Tabor, A. Nowak, A. Bedychaj, M. Patacchiola, T. Trzcinski, P. Spurek, and M. Zieba. Non-gaussian gaussian processes for few-shot regression. *Advances in Neural Information Processing Systems*, 34:10285–10298, 2021.

[45] M. Sendera, M. Przewięźlikowski, K. Karanowski, M. Zięba, J. Tabor, and P. Spurek. Hypershot: Few-shot learning by kernel hypernetworks. *arXiv preprint arXiv:2203.11378*, 2022.

[46] J. Snell and R. Zemel. Bayesian few-shot classification with one-vs-each pólya-gamma augmented gaussian processes. In *International Conference on Learning Representations*, 2020.

[47] J. Snell, K. Swersky, and R. S. Zemel. Prototypical networks for few-shot learning. *arXiv preprint arXiv:1703.05175*, 2017.

[48] F. Sung, Y. Yang, L. Zhang, T. Xiang, P. H. Torr, and T. M. Hospedales. Learning to compare: Relation network for few-shot learning. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1199–1208, 2018.

[49] O. Vinyals, C. Blundell, T. Lillicrap, D. Wierstra, et al. Matching networks for one shot learning. *Advances in neural information processing systems*, 29:3630–3638, 2016.

[50] Y. Wang, Q. Yao, J. Kwok, and L. M. Ni. Generalizing from a few examples: A survey on few-shot learning, 2020.

[51] Z. Wang, Z. Miao, X. Zhen, and Q. Qiu. Learning to learn dense gaussian processes for few-shot learning. *Advances in Neural Information Processing Systems*, 34, 2021.

[52] H. Ye, H. Hu, D. Zhan, and F. Sha. Learning embedding adaptation for few-shot learning. *CoRR*, abs/1812.03664, 2018.

[53] H.-J. Ye and W.-L. Chao. How to train your maml to excel in few-shot classification, 2021.

[54] J. Yoon, T. Kim, O. Dia, S. Kim, Y. Bengio, and S. Ahn. Bayesian model-agnostic meta-learning. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, pages 7343–7353, 2018.

[55] D. Zhao, J. von Oswald, S. Kobayashi, J. Sacramento, and B. F. Grewe. Meta-learning via hypernetworks. 2020.

[56] X. Zhen, Y.-J. Du, H. Xiong, Q. Qiu, C. Snoek, and L. Shao. Learning to learn variational semantic memory. In *NeurIPS*, 2020.

[57] A. Zhmoginov, M. Sandler, and M. Vladymyrov. Hypertransformer: Model generation for supervised and semi-supervised few-shot learning. In *International Conference on Machine Learning*, pages 27075–27098. PMLR, 2022.

[58] F. Zhuang, Z. Qi, K. Duan, D. Xi, Y. Zhu, H. Zhu, H. Xiong, and Q. He. A comprehensive survey on transfer learning, 2020.